



- 伪指令
- 汇编语言的语句格式
- 汇编程序应用
- 汇编语言与 C/C++ 的混合编程

1. 符号定义伪指令

- 符号定义伪指令**作用**：用于定义 ARM 汇编程序中的变量、对变量赋值以及定义寄存器的别名等。

- 符号定义有如下几种伪指令：

定义局部变量：LCLA、LCLL、LCLS

定义全局变量：GBLA、GBLL、GBLS

变量赋值：SETA、SETL、SETS

寄存器列表定义名称：RLIST

(3) 变量赋值

- 格式：

变量名 SETA/SETL/SETS 表达式

- 功能：为变量赋值。

- SETA：给一个数字变量赋值。

- SETL：给一个逻辑变量赋值。

- SETS：给一个字符串变量赋值。

- 格式中的变量名必须为已经定义过的全局或局部变量，表达式为将要赋给变量的值。

- 例如：

```
LCLA num1 ; 定义一个局部的数字变量，变量名为 num1
num1 SETA 0x1234 ; 将该变量赋值为 0x1234
LCLS str3 ; 定义一个局部的字符串变量，变量名为 str3
str3 SETS "Hello!"; 将该变量赋值为"Hello!"
```

- 伪指令：在 ARM 汇编语言程序里，有一些特殊指令助记符，这些助记符与指令系统的助记符不同，**没有相对应的操作码**，通常称这些特殊指令助记符为伪指令。

- 伪操作：伪指令完成的操作即为伪操作。

- 作用：伪指令在源程序中的作用是为完成汇编程序作各种准备工作的，由汇编程序在源程序的汇编期间进行处理，**仅在汇编过程中起作用**。

- 例如：要告诉编译器程序段的开始和结束，需要定义数据等。

- ARM 汇编程序伪指令包括符号定义伪指令、数据定义伪指令、汇编控制伪指令、宏指令以及其他伪指令。

- 参考教材把伪指令分为三部分介绍：通用伪指令、与 ARM 指令相关的伪指令、与 Thumb 指令相关的伪指令。

(1) 定义全局变量

- 格式：

GBLA/GBLL/GBLS 变量名

- 功能：定义一个汇编程序中的全局变量并初始化。

- GBLA 定义一个全局数字变量，并初始化为 0。

- GBLL 定义一个全局逻辑变量，并初始化为 F。

- GBLS 定义一个全局字符串变量，并初始化为空串。

- 这 3 条伪指令用于定义全局变量，因此在整个程序范围内变量名必须惟一。

- 例如：

```
GBLA num1 ; 定义一个全局的数字变量，变量名为 num1
num1 SETA 0xabcd ; 将该变量赋值为 0xabcd
GBLL l2 ; 定义一个全局的逻辑变量，变量名为 l2
l2 SETL {FALSE} ; 将该变量赋值为假
GBLS str3 ; 定义一个全局的字符串变量，变量名为 str3
str3 SETS "Hello!"; 将该变量赋值为"Hello!"
```

(4) 寄存器列表定义

- 格式：

名称 RLIST {寄存器列表}

- 功能：用于对一个通用寄存器列表定义名称，该名称可在 ARM 指令 LDM/STM 中使用。

- 在 LDM/STM 指令中，列表中的寄存器按照寄存器的编号由低到高的次序访问，与列表中的寄存器排列次序无关。

- 例：

```
pblock RLIST {R0-R3, R7, R5, R9}
; 将寄存器列表名称定义为 pblock，可在 ARM
指令 ; LDM/STM 中通过该名称访问寄存器列表
```

- 通用伪指令包括：

符号定义伪指令

数据定义伪指令

汇编控制伪指令

及其他一些常用伪指令等。

(2) 定义局部变量

- 格式：

LCLA/LCLL/LCLS 局部变量名

- 功能：定义一个汇编程序中的局部变量并初始化。

- LCLA 定义一个局部的数字变量，初始化为 0。

- LCLL 定义一个局部的逻辑变量，初始化为 F。

- LCLS 定义一个局部的字符串变量，初始化为空串。

- 这 3 条伪指令用于声明局部变量，在其局部作用范围内变量名必须惟一，例如在宏内。

- 例：

```
MACRO TEST
LCLA num1 ; 定义一个局部的数字变量，变量名为 num1
LCLL l2 ; 定义一个局部的逻辑变量，变量名为 l2
LCLS str3 ; 定义一个局部的字符串变量，变量名为 str3
num1 SETA 0xabcd ; 将该变量赋值为 0xabcd
l2 SETL {FALSE} ; 将该变量赋值为假
str3 SETS "Hello!"; 将该变量赋值为"Hello!"
...
MEND
```

- 在宏内定义局部变量后，则在宏外使用该指令时编译会出错。

2 . 数据定义伪指令

- 作用：为数据分配存储单元，同时初始化。

- 有如下几种：

- DCB 字节分配

- DCW/DCWU 半字 (2 字节) 分配

- DCD/DCDU 字 (4 字节) 分配

- DCQ/DCQU 8 个字节分配

- DCFS/DCFSU 单精度浮点数分配

- DCFD/DCFUD 双精度浮点数分配

- SPACE 分配一块连续的存储单元

- FIELD 定义一个结构化的内存表的数据域

- MAP 定义一个结构化的内存表首地址

(1) DCB

- 格式：
标号 DCB 表达式
- 说明：分配一块字节单元，用表达式值初始化。
- 表达式：使用双引号的字符串，或 0~255 的数字。
- DCB 可用“=”代替。
- 例如：
Array1 DCB 1, 2, 3, 4, 5 ; 数组
str1 DCB "Your are welcome !" ; 构造字符串并分配空间

(2) DCW/DCWU

- 格式：
标号 DCW/DCWU 表达式
- 说明：分配一段半字存储单元，用表达式值初始化。
- 定义的存储空间是半字对齐的。
- DCWU 功能与 DCW 类似，只是分配的字存储单元不严格半字对齐。
- 例如：
Arrayw1 DCW 0xa, -0xb, 0xc, -0xd ; 构造固定数组，分配半字存储单元

(3) DCD/DCDU

- 格式：
标号 DCD/DCDU 表达式
- 说明：分配一块字存储单元，用表达式值初始化。
- 定义的存储空间是字对齐的。
- DCD 也可用“&”代替。
- DCDU 功能与 DCD 类似，只是分配的存储单元不严格字对齐。
- 例如：
Arrayd1 DCD 1334, 234, 345435 ; 构造数组，并分配字存储单元
Label DCD str1 ; 该字单元存放 str1 的地址

(4) DCFD/DCFDU

- 格式：
标号 DCFD/DCFDU 表达式
- 说明：为双精度的浮点数分配一片连续的字存储单元，用表达式值初始化。
- 定义的存储空间是字对齐的。
- 每个双精度的浮点数占据两个字单元。
- DCFDU 功能与 DCFD 类似，只是分配的存储单元不严格字对齐。
- 例如：
Arrayf1 DCFD 6E2
Arrayf2 DCFD 1.23, 1.45

DCFS/DCFSU

- 格式：
标号 DCFS/DCFSU 表达式
- 说明：为单精度的浮点数分配一片连续的字存储单元，用表达式值初始化。
- 定义的存储空间是字对齐的。
- 每个单精度浮点数使用一个字单元。
- DCFSU 功能与 DCFS 类似，只是分配的存储单元不严格字对齐。
- 例如：
Arrayf1 DCFS 6E2, -9E-2, -.3
Arrayf2 DCFSU 1.23, 6.8E9

(5) DCQ/DCQU

- 格式：
标号 DCQ/DCQU 表达式
- 说明：分配一块以 8 个字节为单位的存储区域，用表达式值初始化。
- 定义的存储空间是字对齐的。
- DCQU 功能与 DCQ 类似，只是分配的存储单元不严格字对齐。
- 例如：
Arrayd1 DCQ 234234, 98765541 ; 构造数组，分配字单元存储空间。
; 注意：DCQ 不能给字符串分配空间

(6) MAP

- 格式：
MAP 表达式 [, 基址寄存器]
- 说明：定义一个结构化的内存表的首地址。
- 表达式：程序中的标号，或数学表达式。
- 基址寄存器：可选项，
(1) 基址寄存器不存在时：内存表首地址 = 表达式值。
(2) 选项存在时：内存表首地址 = 表达式值 + 基址寄存器。
- MAP 可以与 FIELD 伪操作配合使用来定义结构化的内存表。
- “^”可以用来代替 MAP。
- 例如：
MAP 0x130, R2 ; 内存表首地址 = 0x130 + R2

FIELD

- 格式：
标号 FIELD 字节数
- 说明：定义一个结构化内存表中的数据域。
- FIELD 常与 MAP 配合使用来定义结构化的内存表：FIELD 定义内存表中的各个数据域，MAP 则定义内存表的首地址，并为每个数据域指定一个标号以供其他的指令引用。
- 需要注意的是 MAP 和 FIELD 伪指令仅用于定义数据结构，并不分配存储单元。
- “#”可用来代替 FIELD。
- 例如：
MAP 0xF1000 ; 定义结构化内存表首地址为 0xF1000
count FIELD 4 ; 定义 count 的长度为 4 字节，位置为 0xF1000+0
x FIELD 4 ; 定义 x 的长度为 4 字节，位置为 0xF1004
y FIELD 4 ; 定义 y 的长度为 4 字节，位置为 0xF1008

(7) SPACE

- 格式：
标号 SPACE 表达式
- 说明：分配一片连续的存储区域，初始化为 0。
- 表达式：要分配的字节数。
- SPACE 也可用“%”代替。
- 例如：
freespace SPACE 1000 ; 分配 1000 字节的存储空间

3 . 汇编控制伪指令

- **作用：指引**汇编程序的执行流程。

- **常用的伪操作包括：**

- (1) **MACRO**、**MEND** 和 **MEXIT**：宏定义的开始与结束、从宏中退出。
- (2) **IF**、**ELSE** 和 **ENDIF**：根据逻辑表达式的成立与否决定是否在编译时加入某个指令序列。
- (3) **WHILE** 和 **WEND**：根据逻辑表达式的成立与否决定是否循环执行这个代码段。

(2) IF、ELSE、ENDIF

- 语法格式：

```
IF 逻辑表达式
    语句段 1
ELSE
    语句段 2
ENDIF
```

- **功能：**当**逻辑表达式 = 真**时，**执行语句段 1**，否则执行语句段 2。
- **ELSE** 及语句段 2 可以没有，此时，若逻辑表达式 = 真，则执行指令序列 1，否则继续执行后面的指令。
- **指令示例：**

```
IF R0=0x10          ; 判断 R0 中的内容是否是 0x10
    ADD R0, R1, R2   ; 如果 R0= 0x10 ，则执行 R0= R1+ R2
ELSE
    ADD R0, R1, R3   ; 如果 R0≠ 0x10 ，则执行 R0= R1+ R3
ENDIF
```

4 . 其他伪指令

- 在汇编程序中经常会使用一些其他的伪指令，包括以下 18 条：

ALIGN	AREA
CODE16/CODE32	ENTRY
END	EQU
GET/INCLUDE	INCBIN
IMPORT	EXPORT

(1) MACRO、MEND 和 MEXIT

- 语法格式：

```
MACRO
[$ label] macroname { $ parameter1 , $ parameter , ..... }
    指令序列
MEND
```

- **宏：**一段**独立的程序代码**，是**通过伪指令定义的**，在程序中使用宏指令即可调用宏。当程序被汇编时，汇编程序将对每个调用进行展开，用宏定义取代源程序中的宏指令。
- **MACRO**：定义一个宏语句段的开始。
- **MEND**：定义宏语句段的结束。
- **MEXIT**：从宏语句段的跳出。
- **\$ label**：可选。在宏指令被展开时，label 会被替换成相应的符号，通常是一个标号。在一个符号前使用 \$ 表示程序被汇编时将使用相应的值来替代 \$ 后的符号。
- **Macroname**：宏名。
- **\$parameter**：宏指令参数。当宏指令被展开时将被替换成相应的值，类似于函数中的形式参数，可以在宏定义时为参数指定相应的默认值。

(3) WHILE、WEND

- 语法格式：

```
WHILE 逻辑表达式
    语句段
WEND
```

- **功能：**当逻辑表达式 = 真，则执行语句段，该语句段执行完毕后，再判断逻辑表达式的值，若为真则继续执行，一直到逻辑表达式的值为假。
- **WHILE**、**WEND** 伪指令是条件循环伪指令，能根据条件的成立与否决定是否循环执行某个语句段。
- **WHILE**：对条件进行判断，满足条件循环，不满足条件结束循环。
- **WEND**：定义循环体结束。

(1) ALIGN

- 格式：

```
ALIGN [表达式] , 偏移量 ]]
```

- **说明：**通过填充字节，使当前的位置满足一定的**对齐方式**。
- **表达式：**表达式的值为 2 的 n 次幂， $0 \leq n \leq 31$ ，如 1、2、4、8、16 等，用于指定对齐方式。
- **没有表达式：**默认，字对齐。
- **偏移量：**数字表达式，自动对齐到“表达式值 + 偏移量”。
- **例如：**

```
B START
ADD R0, R1, R2
DATA1 DCB “abcde”
ALIGN 4
START LDR R5, [R6]
```

宏示例

- **调用宏：**通过调用宏的名称来实现。宏指令的使用方式和功能与子程序有些相似，子程序可以提供模块化的程序设计、节省存储空间并提高运行速度。
- 但在使用子程序结构时需要保护现场，从而增加了系统的开销，因此，在代码较短且需要传递的参数较多时，可以使用宏指令代替子程序。调用宏的好处是不占用传送参数的寄存器，不用保护现场。
- **指令示例：**

```
MACRO                ; 定义宏
SDATA1 MAX $N1, $N2 ; 宏名称是 MAX，主标号是 SDATA1，两个参数
    语句段           ; 语句段
    .....
MEND                ; 宏结束
```

WHILE 示例

- 指令示例：

```
GBLA Cou1            ; 声明一个全局的数字变量 Cou1
Cou1 SETA 1          ; 为 Cou1 赋值 1
WHILE Cou1< 10       ; 判断 WHILE Counter < 10 进入循环
    ADD R1, R2, R3    ; 循环执行语句
    Cou1 SETA Cou1+1  ; 每次循环 Cou1 加 1
WEND                 ; 执行 ADD R1, R2, R3 语句 10 次后，结束循环。
```

- **注意：**用来进行条件判断的逻辑表达式必须是编译程序能够判断的语句，一般应该是伪指令语句。

(2) AREA

- **格式：** AREA 段名 属性，.....
- **功能：**定义段。
- **说明：**如果段名以数字开头，那么该段名需用“|”字符括起来，如 |7wolf|，用 C 的编译器产生的代码一般也用“|”括起来。
- **属性：**表示该代码段 / 数据段的相关属性，多个属性用“,”分隔。
- **常见属性如下：**
 - ① DATA：定义数据段。
 - ② CODE：定义代码段。
 - ③ READONLY：表示本段为只读。
 - ④ READWRITE：表示本段可读写。
 - ⑤ ALIGN= 表达式，表达式的值为 2 的 n 次幂。
 - ⑥ COMMON 属性：定义一个通用段，这个段不包含用户代码和数据。
- **注意：**一个程序**至少包含一个段**，可以有多个数据段，多个代码段。
- **例：** AREA test，CODE，READONLY

(3) CODE16/CODE32

- **格式：** CODE16/CODE32
- **功能：** CODE16 指示编译器后面的代码为 16 位的 Thumb 指令。
CODE32 指示编译器后面的代码为 32 位的 ARM 指令。
- **说明：** 用于汇编源代码中同时包含 Thumb 和 ARM 指令的情况。
- CODE16/CODE32 不能对处理器进行状态的切换。
- **例如：**
CODE32 ; 32 位的 ARM 指令
AREA ||.text||,CODE , READONLY
...
LDR R0 , = 0x8500+1
BX R0 ; 程序跳转, 并将处理器切换到 Thumb 状态
...
CODE16 ; 16 位的 Thumb 指令
ADD R3 , R3 , 1
END ; 源文件结束

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 28

(6) EQU

- **格式：** 名称 EQU 表达式 [, 类型]
- **功能：** 等值。
- **说明：** 将程序中的数字常量、标号、寄存器值等赋予一个等效的名称, 这一点类似于 C 语言中的 # define 。
- 如果表达式为 32 位的常量, 可以指定表达式的数据类型, 类型域可以有以下 3 种: CODE16、CODE32、DATA 。
- **可用“*”代替 EQU 。**
- **例如：**
num1 EQU 1234 ; 定义 num1 为 1234
addr5 EQU str1+0x50
d1 EQU 0x2400 , CODE32
; 定义 d1 的为 0x2400 ,
; 且该处为 32 位的 ARM 指令

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 31

(8) IMPORT

- **格式：**
IMPORT 标号 [, WEAK]
- **功能：** 告诉编译器, 这个标号是在外部定义的, 即在其他的源文件中定义的。
- **[, WEAK]** : 如果所有的源文件都没有找到这个标号的定义, 编译器也不会提示错误信息。
- **例如：**
AREA mycode , CODE , READONLY
IMPORT _printf
; 通知编译器当前文件要引用函数 _printf
...
END

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 34

(4) ENTRY

- **格式：** ENTRY
- **功能：** 指定汇编程序的入口。
- 在一个完整的汇编程序中至少要有个 ENTRY 。
- 程序中也可以有多个, 此时, 程序的真正入口点可在链接时指定。
- 但在一个源文件里最多只能有一个 ENTRY , 也可以没有。
- AREA subrout, CODE, READONLY ; 代码段, 属性为只读
ENTRY ; 标注程序入口
- start
MOV r0 , #10 ; 设置参数
MOV r1 , #3
BL doadd ; 调用子程序
stop
MOV r0 , #0x18 ; 正常返回控制状态的入口参数
LDR r1 , = 0x20026
SWI 0x123456 ; ARM 中断指令 SWI
doadd
ADD r0 , r0 , r1 ; 子程序代码
MOV pc , lr ; 子程序返回
END ; 程序结束

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 29

(7) GET

- **格式：** GET 文件名
- **说明：** 将一个源文件包含到当前的源文件中, 并将被包含的源文件在当前位置展开进行汇编处理。
- **使用方法：** 在某源文件中定义一些宏指令, 用 MAP 和 FIELD 定义结构化的数据类型, 用 EQU 定义常量的符号名称, 然后用 GET 将这个源文件包含到其他的源文件中。
- 使用方法与 C 语言中的“#include”相似。
- GET 只能用于包含源文件, 包含其他文件则需要使用 INCBIN 伪指令。
- **例如：**
AREA mycode,DATA,READONLY
GET E : \code\prog1.s ; 通知编译器在当前源文件
; 包含源文件 E : \code\
prog1.s
GET prog2.s ; 通知编译器当前源文件包含
; 可搜索目录下的 prog2.s
...
END

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 32

(9) EXPORT

- **格式：**
EXPORT 标号 [, WEAK]
- **功能：** 声明一个全局标号, 其他文件可以引用。
- 可以用 GLOBAL 代替 EXPORT 。
- **[, WEAK]** : 可选项, 声明其他文件有同名的标号, 则该同名标号优先于该标号被引用。
- **例如：**
AREA ||.text|| , CODE , READONLY
main PROC
...
ENDP
EXPORT main ; 声明一个可全局引用的函数 main
...
END

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 35

(5) END

- **格式：**
END
- **功能：** 告诉编译器源程序结束。
- **例如：**
AREA constdata , DATA , READONLY
...
END ; 结尾

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 30

INCBIN

- **格式：**
INCBIN 文件名
- **说明：** 将一个数据文件或者目标文件包含到当前的源文件中, 编译时被包含的文件不作任何变动地存放在当前文件中, 编译器从后面开始继续处理。
- **例如：**
AREA constdata , DATA , READONLY
INCBIN data1.dat ; 源文件包含文件 data1.dat
INCBIN E : \DATA\data2.bin ; 源文件包含文件 E :
; \DATA\data2.bin
...
END

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 33

4.1.2 与 ARM 指令相关的伪指令

- 与 ARM 指令相关的伪指令共有 4 条。
- 这 4 条伪指令和通用伪指令不同, 在程序编译过程中, 编译程序会为这 4 条指令产生代码, 但这些代码不是它们自己的代码, 所以尽管它们可以产生代码, 但还是伪指令。
 - ADR 伪指令
 - ADRL 伪指令
 - LDR 伪指令
 - NOP

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 36

1. ADR 小范围地址读取

- 格式：
ADR{<cond>} <Rd> , <expr>
- 功能：基于 PC 相对偏移的地址值 (expr 地址表达式) → Rd。
- 当地址值是非字对齐时，取值范围在 -255~255 字节之间。
- 当地址值是字对齐时，取值范围在 -1 020 ~ 1 020 字节之间。
- 在编译源程序时，ADR 被编译器替换成一条合适的指令。
- 通常，编译器用一条 ADD 指令或 SUB 指令来实现该 ADR 伪指令的功能。若不能用一条指令实现，则产生错误，编译失败。
- 针对 ARM7，对于基于 PC 相对偏移的地址值时，给定范围是相对当前指令地址后两个字处 (因为 ARM7 为三级流水线)。
- 可以用 ADR 加载地址实现查表。
- 例如：
LOOP MOV R1, #0xF0
ADR R2, LOOP ; 将 LOOP 的地址放入 R2, 因为 PC 值为当前指令地址加 8 字节, 所以本 ADR 指令被编译器换成 "SUB R2, PC, #0xc"

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 37

LTORG

- 格式：
LTORG
- 功能：声明一个数据缓冲池 (文字池) 的开始。
- 通常放在无条件跳转指令之后，或者子程序返回指令之后，以免处理器错误地将数据缓冲池中地数据作为指令来执行。
- 例如：
Func1
.....
MOV PC, LR
LTORG
DATA
SPACE 256 ; 从 DATA 标号开始预留 256 字节的内存单元。
END

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 40

1. ADR 伪指令

- 语法规式：
ADR{<cond>} Rd, 语句标号 + 数值表达式
- 功能：把地址加载到低端寄存器中，这个地址必须是基于 PC 相对偏移的地址值。偏移必须向前偏移，偏移量不大于 1KB，且该指令只可以加载字对准的地址。
- Rd：目标寄存器，为 R0 ~ R7。
- 指令示例：
ADR R0, LOOP ; 把 LOOP 处绝对地址加载给 R0
ADR R1, LOOP+0x40*2 ; 把 LOOP+0x40*2 处绝对地址加载给 R1
.....
ALIGN
LOOP ADD R2, R0, R1
.....

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 43

2. ADRL 中等范围地址读取

- 格式：
ADRL {<cond>} <Rd> , <expr>
- 功能：类似于 ADR，但比 ADR 读取更大范围的地址。
- 当地址值是非字对齐时，取值范围在 -64KB ~ 64 KB 之间；
- 地址值是字对齐时，取值范围在 -256KB~256 KB 之间。
- 在编译源程序时，ADRL 被编译器替换成两条合适的指令。
- 若不能用两条指令实现 ADRL 伪指令功能，则产生错误，编译失败。
- 可以用 ADRL 加载地址，实现程序跳转。
- 例如：
ADRL R0, DATA_BUF
ADRL R1, DATA_BUF+80
DATA_BUF
SPACE 100 ; 定义 100 字节缓冲区

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 38

4. NOP 空操作

- 格式：
NOP
- 功能：不产生任何有意义的操作，只是占用一个机器时间。
- NOP 在汇编时将会被替代成 ARM 中的空操作，比如可能为 "MOV R0, R0" 指令等。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 41

2. LDR 伪指令

- 语法规式：
LDR{<cond>} Rd, = 数值表达式 ; 加载数字常量
LDR{<cond>} Rd, = 语句标号 + 数值表达式 ; 加载地址
- 功能：是把一个数字常量或一个地址加载到低端寄存器伪指令。如果所加载的是一个 32 位的数字常量，则编译程序就可以把这条语句编译成一条 MOV 指令，如果不能用 MOV 指令来表达，则编译成一条 LDR 指令。如果所加载的是地址的话，编译程序会把这条语句编译成 LDR 指令。
- 在使用 LDR 指令替代伪指令时，编译程序先把数据 (或地址) 存放在数据缓冲区内，在执行 LDR 指令时，从缓冲区读出这个数据加载到寄存器中去。因此，在使用这条伪指令时，要为程序创建数据缓冲区。
- 指令示例：
LDR R1, =0xFFE ; 加载 0xFFE 到 R1 中
; 汇编器汇编成 MOV R1,#0xFFE
LDR R1, =START ; 加载 START 处的地址到 R1 中

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 44

3. LDR 大范围地址读取

- 格式：
LDR{<cond>} <Rd> , <=expr/label-expr>
- 说明：32 位的立即数或一个地址值 → Rd。
- 在汇编编译源程序时，LDR 伪指令被编译器替换成一条合适的指令。
- 若加载的常数未超出 MOV 或 MVN 的范围，则使用 MOV 或 MVN 指令代替该 LDR 伪指令；
- 否则汇编器将常量放入文字池，并使用一条程序相对偏移的 LDR 指令从文字池读出常量。
- 从 PC 到文字池的偏移量必须小于 4 KB。
- 与 ARM 指令的 LDR 相比，伪指令的 LDR 的参数有 "=" 符号。
- 例如：
LDR R0, =0x12345678 ; 加载 32 位立即数 0x12345678
LDR R0, =DATA_BUF+60 ; 加载 DATA_BUF 地址 +60
...
LTORG ; 声明文字池

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 39

4.1.3 与 Thumb 指令相关的伪指令

- 与 Thumb 指令相关的伪指令共有 3 条，这些伪指令必须出现在 Thumb 程序段。
- ADR 伪指令
- LDR 伪指令
- NOP

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 42

3. NOP

- 语法规式：
NOP
- NOP 伪指令是空操作指令，在汇编时将被编译成一条无效指令，如 MOV R0, R0，占用 32 位代码空间。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 45

4.2 汇编语言的语句格式

- ARM 汇编语言程序一般由几个段组成，每个段均由 **AREA** 伪操作定义。
- 段可以分为多种，如代码段、数据段、通用段。
- 每个段有不同的属性，如代码段的默认属性为 READONLY，数据段的默认属性为 READWRITE。

```
CODE32                ; 32 位的 ARM 指令段
AREA codesecc, CODE, READONLY
                        ; 代码段，名称为 codesecc，
                        ; 属性为只读

main PROC              ; 函数 main
    STMFD    sp!, {lr}    ; 保护现场
    ADR      r0, strhello  ; strhello 的地址→ r0
    BL       _printf      ; 调用 C 运行时库的 _printf 函数打印
                        ; "Hello world!" 字符串
    BL       welcomefun   ; 调用子函数 welcomefun
    LDMFD    sp!, {pc}     ; 恢复现场
strhello
    DCB      "Hello world!\n\0" ; 定义字符串
    ENDP                      ; 函数 main 结束
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 46

地址标号

- 地址标号：代表地址。
- 地址标号分类：
 - 段内标号：地址值在汇编时确定。
 - 段外标号：地址值在链接时确定。
- 在程序段中，标号代表其所在位置与段首地址的偏移量。根据程序计数器（PC）和偏移量计算地址即程序相对寻址。
- 在映像中，标号代表标号到映像首地址的偏移量。映像的首地址通常被赋予一个寄存器，根据该寄存器值与偏移量计算地址即寄存器相对寻址。
- 例如：

```
loop
    SUBS     r0, r0, #1    ; 每次循环使 r0=r0-1
    BNE     loop          ; 跳转到 loop 标号去执行
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 49

变量

- (2) 逻辑变量：用于在程序的运行中保存逻辑值，逻辑值只有两种取值情况：真或假。
 - ✓ 全局逻辑变量使用伪指令 **GBLL** 定义；
 - ✓ 局部逻辑变量使用伪指令 **LCLL** 定义；
 - ✓ 逻辑变量使用伪指令 **SETA** 赋值。
- (3) 字符串变量：用于在程序的运行中保存一个字符串。
 - ✓ 全局串变量使用伪指令 **GBLS** 定义；
 - ✓ 局部串变量使用伪指令 **LCLS** 定义；
 - ✓ 串变量使用伪指令 **SETS** 赋值。
- 串变量需要使用双引号包含。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 52

汇编语言程序的结构

```
welcomefun             ; 子函数
    STMFD    sp!, {lr}  ; 保护现场
    ADR      r0, adrstrarm ; adrstrarm 的地址→ r0
    LDR      r0, [r0, #0] ; strarm 的内容→ r0
    BL       _printf     ; 调用 C 运行时库的 _printf 函数打印
                        ; "Welcome to ARM world!" 字符串
    LDMFD    sp!, {pc}   ; 恢复现场
adrstrarm
    DCB      strarm      ; 保存 strarm 的地址
    AREA    constdatasec, DATA, READONLY, ALIGN=0 ; 数据段，
                        ; 名称为 constdatasec，属性为只读
strarm
    DCB      "Welcome to ARM world!\n\0" ; 存放字符串
    EXPORT   main         ; 标号可被外部调用
    IMPORT   _main        ; 标号由其他文件定义
    IMPORT   _main        ; 标号由其他文件定义
    IMPORT   _printf      ; 标号由其他文件定义
    IMPORT   [LibSSRequest$SarmLib], WEAK ; 标号由其他文件定义
    END                      ; 程序结束
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 47

局部标号

- 在宏中也可以使用局部标号。
- 局部标号：0 ~ 99 的十进制数，可以重复定义。
- 局部标号引用格式：
 - % {F|B} {A|T} N{routname}
 - %：局部标号引用操作
 - F：编译器只向前搜索
 - B：编译器只向后搜索
 - A：编译器搜索宏的所有嵌套层次
 - T：编译器搜索宏的当前层
- 如果 F 和 B 都没有指定，则编译器先向前搜索，再向后搜索。
- 如果 A 和 T 都没有指定，编译器将从宏的当前层次搜索到宏的最高层次，比当前层次低的层次将不再搜索。
- 例如：

```
01 SUBS     r0,r0,#1    ; 每次循环使 r0=r0-1
    BNE     %B01        ; 跳转到 01 标号去执行
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 50

(4) 变量代换

- 这里所说的变量，是相对于汇编程序的“变量”，是用于汇编程序进行处理的，但一旦编译到程序中，则不会改变，成为常量。
- 1) \$+ 字符串变量：在汇编时编译器将该字符串变量的内容代替该串变量。
- 例：

```
LCLS    str1
LCLS    str2
str1     SETS    "book"
str2     SETS    "It is a $str1."
```

- 则在汇编后，str2 的值为“ It is a book ”。
- 2) \$+ 数字变量：编译器会将该数字变量的值转换为十六进制的字符串，并用该十六进制的字符串代换“\$”后的数字变量。
- 例：

```
No1     SETA    10
Str1     SETS    "The number is $No1."
```

- 则汇编后 Str1 的值为“ The number is 0000000A ”。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 53

4.2.1 书写格式

- 格式：
 - [标号] 指令 / 伪指令 [； 语句的注释]
- 格式说明：
 - (1) 标号必须顶格写，前面不能留空格，后面不能有“；”。
 - (2) ARM 汇编器对标识符的大小写敏感。
- 1. 标号
 - (1) 标号是一个符号，可以代表：指令的地址，变量、常量。
 - (2) 标号一般以字母开头，由字母、数字、下划线组成。
 - (3) 当标号代表地址时，可以以数字开头，其作用范围为当前段或者在下一个 ROUT 伪操作之前。
- 2. 指令 / 伪指令
- 指令 / 伪指令是指令的助记符或者定义符，它告诉 ARM 的处理器应该执行什么样的操作，或者告诉汇编程序伪指令语句的伪操作功能。
- 3. 注释
- 汇编器在编译时，当发现一个分号后，把后面的内容解释为注释，不予以编译。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 48

4.2.2 汇编语言中表达式和运算符

- 在汇编语言程序设计中，经常使用各种符号代替地址、变量和常量等，以增加程序的可读性。
- 1. 变量
- 变量形式有 3 种：数字变量、逻辑变量、字符串变量。
- 变量在编译过程中可能被改变。
- (1) 数字变量：用于在程序的运行中保存数字值，数字变量的取值范围不能超过一个 32 位数所能表达的范围。
 - ✓ 全局数字变量使用伪指令 **GBLA** 定义；
 - ✓ 局部数字变量使用伪指令 **LCLA** 定义；
 - ✓ 数字变量使用伪指令 **SETA** 赋值。

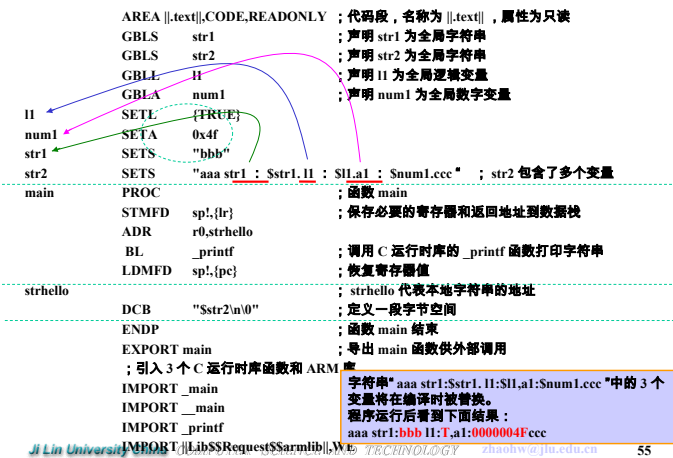
Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 51

汇编程序的变量代换

- 3) \$+\$：此时编译器将不再进行变量代换，而是把“\$\$”看作一个“\$”。
- 例：
 - Str SETS "The character is \$\$"
- 则编译后 Str 字符串的值为“ The character is \$ ”。
- 4) 两个“|”之间的“\$”不进行变量的代换，但如果“|”在双引号内，则将进行变量代换。
- 5) 使用“.”表示字符串中变量名的结束。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 54

变量代换综合示例



数字运算符

- (4) **数字运算符**：表明两个表达式之间的关系。在数字表达式中，运算符有以下几种：
- ① **算术运算符**：+、-、×、/、MOD，比如“A:MOD: B”表示 A 除以 B 的余数。
- ② **移位运算符**：ROL、ROR、SHL、SHR，比如“A:ROL: B”表示将 A 循环左移 B 位。
- ③ **逻辑运算符**：AND、OR、NOT、EOR，比如“A:EOR: B”表示将 A 和 B 按位作逻辑异或的操作。

字符串运算符

单目运算符

单目运算符	语法格式	功能
LEN	:LEN: X	当 X 是一个字符串时，计算 X 的长度。
CHR	:CHR: X	当 X 是一个 0 ~ 255 之间的数字时，把 X 转换成 ASCII 的字符串。
STR	:STR: X	将 32 位的数字表达式 X 转换为 8 个字符的 16 进制字符串，或逻辑表达式 X 转换为字符串 T 或 F。
DEF	:DEF: X	如果符号 A 已经定义，则结果为真，否则为假。

双目运算符

双目运算符	语法格式	功能
LEFT	X :LEFT: Y	从字符串 X 中左侧取字符 Y 个。
RIGHT	X :RIGHT: Y	从字符串 X 中右侧取字符 Y 个。
CC	X :CC: Y	将字符串 Y 接在字符串 X 后面形成一个字符串

常量

- 常量**：其值在程序运行过程中不能被改变。
- 常量分类**：
 - (1) **数字常量**
 - (2) **逻辑常量**
 - (3) **字符串常量**

3. 逻辑表达式及运算符

- 逻辑表达式**：包括逻辑值、逻辑变量、逻辑操作符、关系操作符和括号构成，其表达式的运算结果为真或假。逻辑运算符如下：
- (1) **逻辑值**：只有 { TRUE } 或 { FALSE }。
- (2) **逻辑变量**：可以用伪指令定义逻辑变量。
- (3) **逻辑运算符**：LAND、LOR、LNOT、LEOR 运算符，他们在运算时优先权较低。比如“A:LAND: B”表示将 A 和 B 作逻辑与的操作。
- (4) **关系运算符**：关联的两个操作数必须形式相同，运算结果是一个逻辑值。包括 =、>、<、>=、<=、/=、<> 运算符。
- 指令示例**：
MOV R5, #0xFF00 :MOD: 0xF :ROL: 2
IF R5 :LAND: R6 <= R7
MOV R0, #0x00
ELSE
MOV R0, #0xFF

5. 以寄存器和程序计数器 (PC) 为基址的表达式及运算符

- 常用的与寄存器和程序计数器 (PC) 相关的表达式及运算符如下：
- (1) **BASE 运算符**：返回基于寄存器的表达式中寄存器的编号，其语法格式如下：
:BASE : A
其中，A 为与寄存器相关的表达式。
- (2) **INDEX 运算符**：返回基于寄存器的表达式中相对于其基址寄存器的偏移量，其语法格式如下：
:INDEX : A
其中，A 为与寄存器相关的表达式。

2. 数字表达式及运算符

- 数字表达式**：包括数字、数字常量、数字变量、数字运算符和括号构成。表达式的结果不能超过一个 32 位数的表达范围。
- (1) **数字形式**：十进制、十六进制、N 进制、ASCII
- 十进制**：表达时可以直接表达，默认状态，例如：1234、56789。
- 十六进制**：两种表达方法，一种是在数值前加“0x”，另一种是在数值前加“&”。例如：0x12A、&FF00。
- N 进制**：N 是一个 2~9 之间的整数，表示方法是 n_数值。例如：2_01101111 (二进制数字)、8_54231067 (八进制数字)。
- ASCII**：有些值可以使用 ASCII 表达。例如：‘A’表达 A 的 ASCII 码。属于字符串常量，例如：MOV R1, #‘A’等同于 MOV R1, #0x41。
- (2) **数字常量**：是一个 32 位的整数。可以使用伪指令 EQU 定义一个数字常量，并且定义后不能改变。
- (3) **数字变量**：是被定义为变量的数字。

4. 字符串表达式及运算符

- 字符串表达式**：包括字符串常量、字符串变量、运算符和括号构成。编译器所支持的字符串最大长度为 512 字节。
- (1) **字符串**：用双引号包含在内的一系列字符称为字符串。
- (2) **字符串变量**：被定义为变量的字符串称为字符串变量
- (3) **单目运算符**：是只涉及一个字符串的运算符，在串运算中，单目运算符有较高的优先权。
- (4) **双目运算符**：涉及两个表达式，其中至少有一个是字符串。

6. 运算符的优先级

- 汇编时**运算规则**：
 - (1) 先计算括号内，后计算括号外；
 - (2) 在有多个操作符时，顺序和运算符有关，即按优先级运算；
 - (3) 先单目运算，后双目运算；
 - (4) 在相同优先权情况下，从左到右运算。
- 运算符的优先级如表所示。

运算符	优先级顺序 (由高到底)
单目运算符	↓
乘除和取膜 (×、/、MOD)	
移位运算 (ROL、ROR、SHL、SHR)	
加法和逻辑运算 (+、-、AND、OR、EOR)	
关系运算 (=、>、<、>=、<=、/=、<>)	
逻辑关系运算 (LAND、LOR、LEOR)	

4.3 汇编程序应用

4.3.1 汇编程序基本结构

- 下面是一个汇编语言源程序的基本结构：

AREA example, CODE, READONLY ; 定义代码块为 example
ENTRY ; 程序入口

Start

```
MOV R0, #40 ; R0=40
MOV R1, #16 ; R1=16
ADD R2, R0, R1 ; R2= R0 +R1
MOV R0, #0x18 ; 传送到软件中断的参数
LDR R1, =0x20026 ; 传送到软件中断的参数
SWI 0x123456 ; 通过软件中断指令返回
END ; 文件结束
```

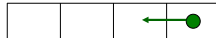
4.3.3 ARM 程序设计

；将 R2 中高 8 位数据传送到 R3 的低 8 位中。

• **MOV R0, R2, LSR #24**



• **ORR R3, R0, R3, LSL #8**



数据变换应用

；已知：R0=A B C D，转换目标：R0= D C B A

• 程序一：

```
EOR R1, R0, R0, ROR #16 ; R1=A^C, B^D, C^A, D^B
BIC R1, R1, #0xFF0000 ; R1=A^C, 0, C^A, D^B
MOV R0, R0, ROR #8 ; R0= D, A, B, C
EOR R0, R0, R1, LSR #8 ; R0= D, C, B, A
```

• 程序二：

```
MOV R2, #0xFF ; R2=0xFF
ORR R2, R2, #0xFF0000 ; R2=0xFF00FF
AND R1, R2, R0 ; R1= 0 B 0 D
AND R0, R2, R0, ROR #24 ; R0= 0 C 0 A
ORR R0, R0, R1, ROR #8 ; R0= D C B A
```

D0B0

BCDA

4.3.2 子程序调用

- 指令格式：**BL** 子程序名
- 该指令在执行时完成如下操作：将子程序的返回地址存放在连接寄存器 **LR** 中，同时将程序计数器 **PC** 指向子程序的入口点。
- 当子程序执行完毕需要返回调用处时，只需要将存放在 **LR** 中的返回地址重新拷贝给程序计数器 **PC**，即：使用指令 **MOV PC, LR**。
- 习惯上用寄存器 **R0 ~ R3** 来存放送到子程序的参数，然后从子程序返回时存放返回的结果给调用者。
- 子程序举例：实现 $R0 = R0 - R1 - R2 - R3$ 。

用移位实现乘法应用

- MOV R0, R0, LSL #n** ; $R0=R0<<n; R0=R0\times(2^{*}n)$
- ADD R0, R0, R0, LSL #n** ; $R0=R0+R0\times(2^{*}n)$
- RSB R0, R0, R0, LSL #n** ; $R0= R0\times(2^{*}n) - R0$

条件执行举例

；求最大公约数

• C 语言代码：

```
int gcd (int a, int b)
{ while (a != b)
    if ( a > b )
        a = a - b;
    else
        b = b - a;
return a;
}
```

• 对应的 ARM 代码段：R0=a，R1=b

```
gcd
CMP R0, R1 ; 比较 a 和 b 大小
SUBGT R0, R0, R1 ; if (a>b) a=a-b (if a==b do nothing)
SUBLT R1, R1, R0 ; if (b>a) b=b-a (if a==b do nothing)
BNE gcd ; if (a!=b) then 跳转到 gcd 处继续执行
MOV PC, LR ; 子程序结束，返回
```

子程序举例

```
AREA Init, CODE, READONLY ; 定义一个代码段
ENTRY ; 定义一个程序入口
LOOP1 MOV R0, #211 ; 给参数 R0 赋值 211
MOV R1, #106 ; 给参数 R1 赋值 106
MOV R2, #64 ; 给参数 R2 赋值 64
MOV R3, #5 ; 给参数 R3 赋值 5
BL SUB1 ; 调用子程序 SUB1，同时将子程序的返回地址
; 存放在链接寄存器 R14 ( LR ) 中。
MOV R0, #0x18 ; 传送到软件中断的参数
LDR R1, =0x20026 ; 传送到软件中断的参数
SWI 0x123456 ; 通过软件中断指令返回
SUB1 SUB R0, R0, R1 ; 子程序代码
SUB R0, R0, R2
SUB R0, R0, R3
MOV PC, LR ; 从子程序返回
END
```

64 位数据加减运算应用

；假设，R1R0 存放一个 64 位数据，R3R2 存放另一个 64 位数据。

- 64 位数据加法运算：R1R0←R1R0 + R3R2
- ADDS R0, R0, R2
- ADC R1, R1, R3

- 64 位数据减法运算：R1R0←R1R0 - R3R2
- SUBS R0, R0, R2
- SBC R1, R1, R3

条件判断语句

• C 语言代码：

```
if ( a == 0 || b == 1 )
    c = d + e ;
```

• 对应的 ARM 代码段：

```
CMP R0, #0 ; 判断 R0 是否等于 0
CMPNE R1, #1 ; 如果 R0 不等于 0，判断 R1 是否
; 等于 1
ADDEQ R2, R3, R4 ; R0 = 0 或 R1 = 1 时，R2 = R3
+ R4
```


循环语句

```
MOV R0, #loopcount
loop
.....
SUBS R0, R0, #1
BNE loop
```

子程序地址及其计算

• 3、子程序地址表（散列表）

JumpTable

```
DCD DoAdd ; 定义 DoAdd 地址
DCD DoSub ; 定义 DoSub 地址
```

• 4、根据编号计算转移地址，并转移

arithfunc

```
CMP r3, #num ; num 是子程序数，R3 是调用的子程序编号
MOVHS pc, lr ; HS 无符号大于等于
ADR r4, JumpTable ; 装载地址表首地址
LDR pc, [r4, r3, LSL#2] ; 跳转到相应子程序入口地址处
```

- 注意：转移地址 = $r4 + r3 \times 4$ ，表中使用 4 字节保存子程序地址。

字符串拷贝程序（用 LDR 和 STR 实现）

	AREA StrCopy, CODE, READONLY	
	ENTRY ; 程序入口	
start		
	LDR r1, =srcstr ; 初始串的指针	
	LDR r0, =dststr ; 结果串的指针	
	BL strcopy ; 调用子程序执行复制	
stop		
	MOV r0, #0x18 ; 执行中止	
	LDR r1, =0x20026	
	SWI 0x123456	
strcopy		
	LDRB r2, [r1], #1 ; 加载并且更新源串指针	
	STRB r2, [r0], #1 ; 存储且更新目的串指针	
	CMP r2, #0 ; 为 0，字符串结束	
	BNE strcopy	
	MOV pc, lr	
srcstr	AREA Strings, DATA, READWRITE	
	DCB "First string - source", 0	
dststr	DCB "Second string - destination", 0	
	END	

完整汇编程序示例

- 例：用查表和散转方法实现：

给定 0，查表转移到 DoAdd 子程序。

给定 1，查表转移到 DoSub 子程序。

- 查表和散转方法：

(1) 将所有子程序起始地址，存到一张表内，并编序号。

(2) 计算转移地址 = 表首址 + 编号 × 子程序地址占用的存储字节数。

(3) 实现转移：转移地址 → PC。

- 程序结构：

(1) 程序框架

(2) 子程序 DoAdd

(3) 子程序 DoSub

(4) 子程序地址表

(5) 计算转移地址并转移

子程序地址表

表首址	子程序0 地址	编号0
	子程序1 地址	编号1
	子程序2 地址	编号2
	子程序3 地址	编号3

完整汇编程序

	AREA Jump, CODE, READONLY	
	CODE32	
	EQU 2	
	ENTRY ; 跳转表的入口数目	
	ENTRY ; 程序入口	
num		
start		
	MOV r0, #0 ; 传递给子程序的参数	
	MOV r1, #3 ; 传递给子程序的参数	
	MOV r2, #2 ; 传递给子程序的参数	
	MOV R3, #0 ; 调用散列表中的子程序序号	
	BL arithfunc ; 调用计算地址的子程序	
stop		
	MOV r0, #0x18 ; 执行中止	
	LDR r1, =0x20026	
	SWI 0x123456	
arithfunc		
	CMP r3, #num ; 比较参数	
	MOVHS pc, lr ; HS 无符号大于	
	ADR r4, JumpTable ; 装载地址表首地址	
	LDR pc, [r4, r3, LSL#2] ; 跳转到相应子程序入口地址	
JumpTable		
	DCD DoAdd	
	DCD DoSub	
DoAdd		
	ADD MOV r0, r1, r2 ; =0 时的操作	
	pc, lr ; 返回	
DoSub		
	SUB MOV r0, r1, r2 ; =1 时的操作	
	pc, lr ; 返回	
	END ; 程序结束	

多字数据拷贝

- 例：将数据串 1 中的数据拷贝到串 2，多字拷贝，提高效率。

- 多字拷贝指令：LDM，STM

- 方法：选用 8 字拷贝，地址变换规则 IA（后加）

- 算法

(1) 获取 8 字的拷贝次数

8 字为单位的拷贝次数：MOVS r3, r2, LSR #3 ; r2 源串字数

从源串读取多字：LDMIA r0!, {r4-r11}

将多字写入目的串：STMIA r1!, {r4-r11}

(2) 剩余以字节拷贝

字为单位的拷贝次数：ANDS r2, r2, #7 ; r2 源串字数

从源串读取 1 个字：LDR r3, [r0], #4

将 1 个字写入目的串：STR r3, [r1], #4

程序框架和子程序

• 1、程序框架

	AREA Jump, CODE, READONLY	
	CODE32 ; ARM 指令	
	ENTRY ; 程序入口	
Start		
stop		
	MOV r0, #0x18 ; 执行中止	
	LDR r1, =0x20026	
	SWI 0x123456 ; ARM semihosting SWI	
	END ; 程序结尾	

• 2、子程序

(1) DoAdd

```
ADD r0, r1, r2
MOV pc, lr ; 子程序返回
```

(2) DoSub

```
SUB r0, r1, r2
MOV pc, lr ; 子程序返回
```

字符串拷贝举例

- 例：将串 1 中的字符数据拷贝到串 2，按字节拷贝。

- 字符串定义指令 DCB，以 0x00 为结束符。

- 字节拷贝指令 LDRB、STRB。

- 拷贝方法

(1) 初始化：

R1 ← 源串，R0 ← 目的串

(2) 拷贝一个字节：

```
LDRB R2, [R1]
```

```
STRB R2, [R0]
```

(3) R1、R0 指向下一个要拷贝的字节数据：

```
ADD R1, R1, #1
```

```
ADD R0, R0, #1
```

(4) 判断字符串是否结束：

```
判断 CMP R2, #0x00
```

```
转移 BNE
```

多字节数据拷贝程序（用 LDM 和 STM 实现）

	AREA Block, CODE, READONLY	
	EQU 20 ; 被拷贝的数据字数	
	ENTRY ; 程序入口	
num		
start		
	LDR r0, =src ; r0 = 源串指针	
	LDR r1, =dst ; r1 = 目的串指针	
	MOV r2, #num ; r2 = 拷贝字数	
	MOV sp, #0x400 ; 设置堆栈指针 (r13)	
blockcopy		
	MOVS r3, r2, LSR #3 ; 字数 / 8	
	BEQ copywords ; 少于 8 个字，转	
	STMFD sp!, {r4-r11} ; 入栈保护	
octcopy		
	LDMIA r0!, {r4-r11} ; 从源串加载 8 个字	
	STMIA r1!, {r4-r11} ; 放入目的串	
	SUBS r3, r3, #1 ; 8 倍字数减 1	
	BNE octcopy ; 未完，继续	
	LDMFD sp!, {r4-r11} ; 出栈恢复	

copywords			
	ANDS	r2, r2, #7	; 奇数字被拷贝
	BEQ	stop	; 为 0 , 转
wordcopy			
	LDR	r3, [r0], #4	; 从源串加载一个字且指针自增
	STR	r3, [r1], #4	; 存储到目的串
	SUBS	r2, r2, #1	; 字数减 1
	BNE	wordcopy	; 未完, 继续
stop			
	MOV	r0, #0x18	; 执行中止
	LDR	r1, =0x20026	
	SWI	0x123456	
AREA BlockData, DATA, READWRITE			
src	DCD	1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8,1,2,3,4	
dst	DCD	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0	
	END		
Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 82			

2. 数据栈的使用规则

- APCS 规定：数据栈为满减类型，对数据栈的操作是 8 字节对齐的。
- 相关名词：
 - (1) 数据栈指针：stack pointer，指向最后一个写入栈的数据的内存地址。
 - (2) 数据栈基地址：stack base，是指数据栈的最高地址。由于 APCS 中的数据栈是 FD 类型的，实际上数据栈中最早入栈数据占据的内存单元是基地址的下一个内存单元。
 - (3) 数据栈界限：stack limit，是指数据栈中可以使用的最低的内存单元地址。
 - (4) 已占用的数据栈：used stack，是指数据栈的基地址和数据栈栈指针之间的区域，其中包括数据栈栈指针对应的内存单元。
 - (5) 数据栈中的数据帧：stack frames，是指在数据栈中，为子程序分配的用来保存寄存器和局部变量的区域。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 85

4.4.1 在 C/C++ 程序中内嵌汇编指令的语格式

- 在 ARM 的 C 语言程序中可以使用关键字 __asm 来加入一段汇编语言的程序。

- 格式：

```
__asm
{
    instruction/*comment*/
    ...
}
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 88

4.4 汇编语言与 C/C++ 的混合编程

- 汇编语言与 C/C++ 的混合编程通常有 3 种方式：
 - (1) 嵌入汇编：在 C/C++ 代码中嵌入汇编指令。
 - (2) 变量互访：在汇编程序和 C/C++ 的程序之间进行变量的互访。
 - (3) 相互调用：汇编程序、C/C++ 程序间的相互调用。
- APCS：ARM Procedure Call Standard，ARM 过程调用标准，提供了紧凑的编写例程的一种机制。
- APCS 规定了子程序调用的基本规则，这些规则包括：
 - (1) 确定寄存器使用
 - (2) 数据栈的使用
 - (3) 参数传递规则

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 83

3. 参数的传递规则

- 根据参数个数是否固定，可以将子程序分为：参数个数可变的子程序、参数个数固定的子程序。
 - (1) 参数个数可变的子程序
- 参数传递规则：参数 ≤ 4，使用 R0~R3 进行参数传递，当参数超过 4 个时，使用数据栈传递参数。在参数传递时，将所有参数看做是存放在连续的内存单元中的字数据。然后，依次将各名字数据传送到寄存器 R0、R1、R2、R3；如果参数多于 4 个，将剩余的字数数据传送到数据栈中，入栈的顺序与参数顺序相反，即最后一个字数数据先入栈。
- (2) 参数个数固定的子程序
- 参数传递规则：对于浮点运算，各个浮点参数按顺序处理，第一个整数参数通过寄存器 R0~R3 来传递，其他参数通过数据栈传递。
- 要点：如果函数有 4 个参数，则将分别用 r0、r1、r2 和 r3 来传递，如果参数多于 4 个，则多余的参数将被压入堆栈。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 86

C 中嵌入 ARM 汇编需要注意的问题

- (1) 在汇编指令中，可以使用表达式。
- (2) 一条指令占用多行，使用符号“\”续行。
- (3) 一行有多个汇编指令，使用“;”将多个指令隔开。
- (4) 注释：使用 C 语言中的“/* */”或者“//”进行注释，不再使用“;”。
- (5) 可以使用关键词 asm 来内嵌一段汇编程序，其格式如下：

```
asm (“instruction [; instruction]”);
```

其中，asm 后面括号中必须是一条汇编语句，且其不能包含注释。
- (6) 内嵌汇编用到的寄存器，编译器自动保护、恢复寄存器值，除了 SPSR、SPSR 外，其他寄存器必须先赋值后读取。
- (7) 注意：C 语言变量不要与 ARM 寄存器重名。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 89

1. 各寄存器的使用规则

- (1) R0~R3 用于传递参数：子程序通过寄存器 R0~R3 来传递参数，记作 A0~A3，被调用的子程序在返回前无需恢复寄存器 R0~R3 的内容。
- (2) R4~R11 用于保存局部变量：记作 V1~V8，子程序进入时必须保存这些寄存器的值，在返回前必须恢复这些寄存器的值。
- (3) R12 用于子程序间 scratch 寄存器：记作 ip，在子程序的连接代码段中经常会有这种使用规则。
- (4) R13 用于数据栈指针：记做 SP，在子程序中 R13 不能用做其他用途。寄存器 SP 在进入子程序时的值和退出子程序时的值必须相等。
- (5) R14 用于链接寄存器：记作 lr，用于保存子程序的返回地址，如果在子程序中保存了返回地址，则 R14 可用作其它的用途。
- (6) R15 用于程序计数器：记作 PC，它不能用作其他用途。
- (7) APCS 中的各寄存器在 ARM 编译器和汇编器中都是预定义的。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 84

4. 子程序结果返回规则

- 子程序结果返回规则：
 - (1) 结果为一个 32 位的整数时，通过寄存器 R0 返回。
 - (2) 结果为一个 64 位整数时，可以通过 R0 和 R1 返回，依此类推。
 - (3) 对于位数更多的结果，需要通过调用内存来传递。
- 实际编程应用中，使用较多的方式是：
 - (1) 用汇编语言完成程序的初始化。
 - (2) 用 C/C++ 完成主要的编程任务。
- 程序在执行时首先完成初始化过程，然后跳转到 C/C++ 程序代码中，汇编程序和 C/C++ 程序之间一般没有参数的传递，也没有频繁的相互调用，因此，整个程序的结构显得相对简单，容易理解。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 87

4.4.2 C/C++ 与汇编语言的混合编程应用

- 1. 在 C 语言中内嵌汇编
- 在 C 中内嵌的汇编指令存在一些限制：
 - (1) 不能直接向 PC 寄存器赋值，程序跳转要使用 B 或者 BL。
 - (2) 不要使用过于复杂的 C 表达式，避免物理寄存器冲突。
 - (3) R12 和 R13 可能被编译器用来存放中间编译结果，计算表达式值时可能将 R0~R3、R12 及 R14 用于子程序调用，因此要避免直接使用这些物理寄存器。
 - (4) 一般不要直接指定物理寄存器，而让编译器进行分配。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 90

在 C 中内嵌汇编语言举例

```
#include <stdio.h>
void my_strcpy (const char *src , char *dest) // 声明一个函数
{
    char ch; // 声明一个字符型变量。
    __asm // 调用关键词 __asm 。
    {
        LOOP // 循环入口。
        LDRB ch , [src] , #1 // ARM 指令, ch←[src] , src←src+1。
        STRB ch , [dest] , #1 // ARM 指令, [dest]←ch , [dest]←[dest]
        +1。
        CMP ch , #0 // 比较 ch 是否为零, 判断字符串是否结束。
        BNE LOOP // NE 为不相等条件, ch 是否
        零, 否则循环。
    }
}
```

注意：教材此处有 2 个问题，（1）C 和 ARM 对字母大小写敏感；（2）语句解释错误。

3. C 程序中调用汇编的函数

- C 文件 strtst.c
#include <stdio.h>
extern void strcpy(char *d, const char *s); // 标号来自外部

r0 r1

参数传递：从左~右→从 r0 ~ r3, 多于 4 个参数时, 多于部分压入堆栈。

```
int main()
{
    const char *srcstr = "First string - source\0"; // 定义源串
    char dststr[] = "Second string - destination\0"; // 定义目的串
    printf("Before copying:\n"); // 显示串拷贝前的提示信息
    printf(" '%s'\n '%s'\n", srcstr, dststr); // 显示串拷贝前的数据
    strcpy(dststr, srcstr); // 调用汇编语言编写的串拷贝子程序
    printf("After copying:\n"); // 显示串拷贝后提示信息
    printf(" '%s'\n '%s'\n", srcstr, dststr); // 显示串拷贝后的
    结果
    return 0;
}
```

C 语言主程序

```
int main () ; C 语言主程序
{
    char *a = "forget it and move on!"; // 声明字符型指针变量
    char b[64]; // 字符型数组
    my_strcpy (a , b); // 调用函数, 进行复制
    printf ("original: %s" , a); // 屏幕输出, a 的数值
    printf ("copyed: %s" , b); // 屏幕输出, b 的数值
    return 0;
}
```

C 程序调用汇编程序

- 汇编文件 scopy.s
AREA SCopy, CODE, READONLY
EXPORT strcpy
strcpy
extern void strcpy(char *d, const char *s)
参数传递：从左~右→从 r0 ~ r3, 多于 4 个参数时, 多于部分压入堆栈。
LDRB r2, [r1], #1 ; 读取字节并修改地址
STRB r2, [r0], #1 ; 存储字节并修改地址
CMP r2, #0 ; 检查读取的字节是否为 0
BNE strcpy ; 不为 0 继续
MOV pc, lr ; 否则返回
END

2. 在汇编中使用 C 程序全局变量

C 语言代码文件 str.c 里面只有一个简单的字符串的定义：

```
char *strhello = "Hello world!\n\0" ;
```

汇编代码文件 hello.s

```
1 AREA ||.text||, CODE, READONLY
```

```
2 main PROC
```

```
3 STMFD sp!, {lr}
```

```
4 LDR r0, =strtemp
```

```
5 LDR r0, [r0] ←
```

读取 C 变量的地址

```
6 BL _printf
```

```
7 LDMFD sp!, {pc}
```

```
8 strtemp
```

```
9 DCD strhello ←
```

存储 C 变量的地址

```
10 ENDP
```

```
11 EXPORT main
```

```
12 IMPORT strhello
```

IMPORT : 变量由外部程序定义

```
13 IMPORT _main
```

```
14 IMPORT _main
```

```
15 IMPORT _printf
```

```
16 IMPORT ||Lib$$Request$$armlib||, WEAK
```

[.WEAK] 指示编译器, 如果发现访问的名称在所有的源文件中都没有找到, 那么它也不会产生任何的错误信息。

第 4 章 结束